AIStudio — Codebase Guide

Version: Beta | Updated: 2026-06-15

Purpose

This document provides a structured introduction to the AIStudio codebase — for a new developer, a technical reviewer, or an AI agent bootstrapping context. It describes the general architecture, the key decisions made, and the first two layers of the directory hierarchy. It does not describe every file in detail; that is what the code itself is for.

General Principles

AIStudio follows conventional Python project structure with deliberate discipline around a few non-standard choices:

Standard practices adhered to: - `src/` layout — all application code lives under `src/`, keeping the repo root clean - `pyproject.toml` as the single project config file (ruff, pytest settings) - `tests/` at repo root — test discovery is unambiguous - Pre-commit hooks (ruff lint + format) enforced on every commit - GitHub Actions CI runs the full test suite on every push - Conventional commits in practice

Non-standard decisions (intentional): - A single HTML file (`front_end/rag_studio.html`) is the entire frontend — no build step, no node_modules, no bundler. All JS and CSS are inline. AIStudio runs fully offline and the UI must be self-contained. - `data/corpora/` is partially tracked — the shipped `demo/` corpus is tracked in git. The `help/` corpus config file (`help_corpus_metadata.yaml`) is tracked; its PDFs are regenerated at startup. User-generated corpora are gitignored. - No ORM, no database — the application uses Qdrant (vector store) and flat JSONL files for corpus metadata. The data model is intentionally simple for a local, single-user application. - `ais_install` is manifest-driven — `ais_user_commands_manifest.yaml` (repo root) is the source of truth for command paths and

installation type. Adding a new command requires a manifest entry; `ais_install [cmd]` then installs it without touching other aliases.

File Versioning Conventions

AIStudio source files carry an explicit version header. The convention differs by file type.

Shell scripts (`.sh`)

Two lines, both required, both must match:

```
# Version: X.Y.Z
VERSION="X.Y.Z"
```

Python files (`.py`)

Header comment block at the top of the file, before all imports:

```
# Version: X.Y.Z
# Changelog: X.Y.Z — <description of change.>
#            Continuation lines indented 12 spaces (aligned with description start).
# Changelog: X.Y.Z-1 — previous change (newest first).
```

Rules: - `# Version:` must always match the most recent `# Changelog:` entry — it is the single source of truth for the current version. - Changelog entries are prepended (newest first). - One entry per version bump. Two tickets in one bump is acceptable; document both in the same entry. - The header block appears before `from __future__ import annotations` and all other imports.

Architecture Overview

AIStudio is a local RAG (Retrieval-Augmented Generation) application for Apple Silicon. The architecture has three layers:

Ingest layer — documents are loaded, chunked, embedded, and stored in Qdrant. This is a one-time operation per document, triggered via the UI. The ingest pipeline lives in `src/local_llm_bot/app/ingest/` .

Query layer — a FastAPI backend receives questions, retrieves relevant chunks from Qdrant, reranks them with a CrossEncoder, assembles a prompt, and sends it to an Ollama-hosted LLM.

Citations are extracted from the response and returned to the frontend. The query pipeline lives in `src/local_llm_bot/app/rag_core.py` and is orchestrated by `api.py`.

UI layer — a single HTML file provides the complete user interface: corpus management, file upload, chat, settings, and citations rendering. It communicates with the FastAPI backend over localhost.

The three services that must be running are: **Qdrant** (vector store), **Ollama** (LLM host), and the **FastAPI backend** (uvicorn). `ais_start` starts all three.

First-Level Directory Structure

```
AIStudio/
├── src/            Application source code
├── tests/          Test suite
├── front_end/      Single-file frontend (rag_studio.html)
├── data/           Corpus data (partially tracked)
├── docs/           Developer documentation and help corpus sources
├── benchmarks/     Benchmark harness and reports
├── prompts/        LLM system prompt
└── [root files]    Install scripts, user aliases, README, HOWTO, config
```

Second-Level Structure

`src/local_llm_bot/app/` — Core Application

Path	What it is
`api.py`	FastAPI application — all HTTP endpoints: `/ask`, `/corpus/*` (create, rename, delete, info, upload, ingest), `/health`, `/debug/*`, `/source`, `/prewarm`
`rag_core.py`	RAG query pipeline — embed query → Qdrant retrieve → CrossEncoder rerank → Ollama generate → extract citations
`config.py`	Application config — RagConfig, IngestConfig, OllamaConfig, loaded from environment variables
`ollama_client.py`	HTTP client wrapper for the Ollama API
`debug_stats.py`	JSONL stats computation for the `/debug/stats` endpoint

Path	What it is
`ingest/`	Ingest pipeline — loaders, chunking, embedding, Qdrant write
`vectorstore/`	Vectorstore adapters — `qdrant_store.py` (active)
`utils/`	Shared utilities — `corpus_paths.py` (artifact paths), `repo_root.py` (repo discovery)

`src/local_llm_bot/app/ingest/` — Ingest Pipeline

Path	What it is
`__main__.py`	Entry point — `python -m local_llm_bot.app.ingest`
`pipeline.py`	Orchestrates: load → chunk → embed → store
`loaders.py`	File loaders — PDF (pdfplumber), HTM, DOCX, XLSX, MD
`chunking.py`	Semantic chunking — page-aware, respects sentence/paragraph boundaries
`types.py`	Shared types — `Document`, `Chunk`, `RetrievedDoc`
`index_jsonl.py`	`index.jsonl` read/write — chunk metadata on disk
`manifest.py`	`manifest.jsonl` tracking — which files have been ingested

`data/corpora/<name>/` — Corpus Structure

Each corpus has a consistent on-disk layout:

Path	What it is
`uploads/`	Source files uploaded by the user
`trash/`	Files removed from corpus (recoverable) — sibling of uploads/, never inside it
`index.jsonl`	Chunk metadata index
`manifest.jsonl`	File tracking manifest
`doc_chunk_map.json`	Maps documents to their chunk IDs
`{name}_corpus_metadata.yaml`	Search routing guidance — loaded into system prompt at query time

Tracked in git: Only `data/corpora/demo/` (full uploads tracked — ships with repo) and `data/corpora/help/help_corpus_metadata.yaml` (config only — PDFs are regenerated at startup). All other corpus data is gitignored.

`docs/` — Documentation

User-facing and developer documentation. All docs with a PDF companion are part of the built-in help corpus.

File	What it is
`architecture_decisions.md/pdf`	Architecture Decision Records — Qdrant, CrossEncoder, chunking, citation design
`DEMO_CORPUS.md/pdf`	Demo corpus content description
`HARNESS.md/pdf`	Benchmark harness documentation
`PRODUCT_ROADMAP.md/pdf`	PM-facing roadmap — Beta, v2.0, v3.0
`QA_TESTING_LESSONS_LEARNED.md/pdf`	Install and QA testing findings
`dependencies.md/pdf`	Python and system dependency list
`CODEBASE_GUIDE.md/pdf`	This document — codebase introduction

`[root files]` — Install Scripts and User Commands

File	What it is
`ais_install`	Manifest-driven user command installer — `ais_install [cmd]` installs one alias; `ais_install` installs all
`install.sh`	First-time bootstrap — sets up Python venv, installs deps, calls `ais_install`
`ais_start.sh`	Start all services (Qdrant, Ollama, FastAPI, opens UI) — aliased as `ais_start`
`ais_stop.sh`	Stop all services — aliased as `ais_stop`
`ais_bench.sh`	Run benchmark on demo corpus

File	What it is
`ais_log.sh`	Tail live backend log — aliased as `ais_log`
`ais_download_sec_10k.sh`	Download SEC 10-K corpus from EDGAR
`ais_ingest_sec_10k.sh`	Ingest SEC 10-K corpus into AIStudio
`ais_help.sh`	Show user command reference

All `ais_*` commands are installed via `ais_install`, which reads the command manifest to generate `~/.zshrc` aliases — the manifest is the single source of truth for command routing and installation.

How a corpus is defined, built, and used

A *corpus* is a named, queryable collection of documents (internally, a Qdrant collection named `aistudio_<name>`). AIStudio's four built-in corpora come into existence in different ways, and the same patterns let you build your own.

There are three kinds of corpus:

Kind	Examples	What you do to get it
Ships built	`demo`, `help`	Nothing — they're ready on first launch.
Ships with tooling	`sec_10k`, `esef_banks`	Run the bundled downloader to fetch the source documents, then ingest.
Bring your own	*(your corpus)*	Create it in the UI and upload your files.

The three things every corpus has

Whatever kind it is, a built corpus resolves to the same shape under `data/corpora/<name>/`:

1. **The documents** — `data/corpora/<name>/uploads/` (the files that get chunked and embedded).
2. **The corpus metadata** — `data/corpora/<name>/<name>_corpus_metadata.yaml` (its description, search guidance, and default query settings; read every time you query).
3. **(Optional) benchmark questions** — `benchmarks/<name>/<name>_questions.yaml`.

The kinds differ only in how those get populated.

Ships built (`demo`, `help`)

These ship inside the repo and index themselves the first time you run `ais_start` (you'll see a one-time "indexing… background" message). After that, launching is instant and the corpus is just there to query. You don't manage their files.

Ships with tooling (`sec_10k`, `esef_banks`)

These are large public-filing collections, so AIStudio ships the *tools* to fetch them rather than the files themselves. You run the bundled downloader (it knows which firms to fetch), then ingest the results through the UI. `sec_10k` pulls 10-K annual reports from SEC EDGAR; `esef_banks` pulls European bank annual reports from filings.xbrl.org. Once ingested they query like any other corpus. *(These two corpora also resolve each company's legal identity against the GLEIF registry and expand financial terminology so cross-firm questions work — the full walkthrough is in Tutorial Annex 1.)*

Bring your own (your corpus)

You build a corpus the same way the built-in ones look at runtime, but through the UI:

1. **Settings** → **New Corpus** creates `data/corpora/<your-corpus>/` and its `uploads/` folder.
2. The **Upload** button (or drag-and-drop) puts your files into `data/corpora/<your-corpus>/uploads/` and ingests them.
3. The **Edit Corpus** modal sets the corpus's description, search guidance, and default query settings — these are saved to `data/corpora/<your-corpus>/<your-corpus>_corpus_metadata.yaml`.
4. *(Optional)* you can add your own benchmark questions at `benchmarks/<your-corpus>/<your-corpus>_questions.yaml`.

So if you want to query your own portfolio — your filings, reports, decks, or PDFs — drop them in via Upload and the corpus is defined by what's in its `uploads/` folder plus the description you give it in the UI. Nothing else is required.

Using a corpus

When you ask a question, AIStudio reads the corpus's metadata (so your search guidance shapes the answer), retrieves the most relevant chunks from that corpus, and answers with citations back to the source files. This works identically no matter which kind of corpus you're querying.

Key Design Decisions

Why a single HTML file for the frontend? AIStudio runs fully offline. A build step would add complexity and a node_modules dependency. A single file can be opened directly from disk and requires no server to serve static assets.

Why Qdrant over ChromaDB? ChromaDB was the original vectorstore. Qdrant was chosen for its stability on Apple Silicon, its clean REST API, its WAL-based durability, and its superior performance at scale. The migration is complete; ChromaDB code is legacy and unused.

Why CrossEncoder reranking? Embedding-based retrieval is fast but imprecise — it retrieves semantically similar chunks, not necessarily the most relevant ones. A CrossEncoder (ms-marco-MiniLM-L-6-v2) reranks the top-K retrieved chunks against the query, improving answer quality significantly with minimal latency overhead.

Why `{corpus_name}_corpus_metadata.yaml` ? Each corpus has different content and requires different retrieval guidance. The corpus meta YAML is loaded into the system prompt at query time, allowing per-corpus routing instructions without code changes.

Command Tiers

AIStudio commands come in two tiers.

User commands (`ais_*`) are git-tracked, ship with the public repo, installed by `ais_install`, and visible via `ais_help`. These are the commands any developer who clones the repo can use. The full list is in `ais_user_commands_manifest.yaml` at repo root.

Internal tooling handles corpus regeneration, session management, and deployment — these are not part of the public repo and not needed for normal AIStudio use.

User testing: `ais_bench` is the quality validation tool — run benchmark questions against a corpus to measure retrieval accuracy.

★★★ ★★★